



GPT



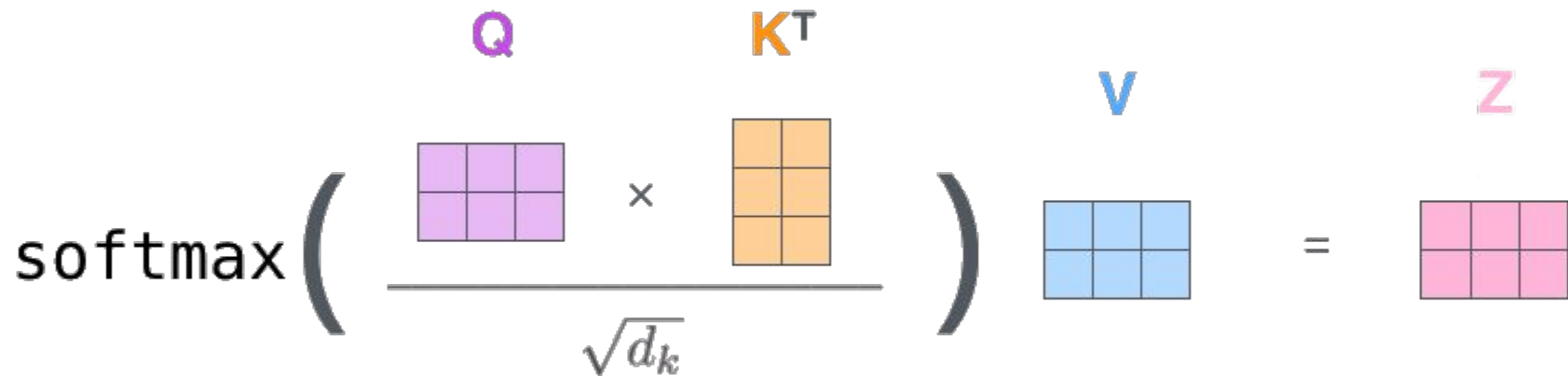
Attention

Q(Query) : (디코더에서) 현재 단어가 무엇을 찾고 있는가?

K(Key) : (인코더의) 단어가 어떤 정보를 갖고 있는가?

V(Value) : (인코더의) 각 단어가 갖고 있는 정보

Scaled Dot-Product Attention

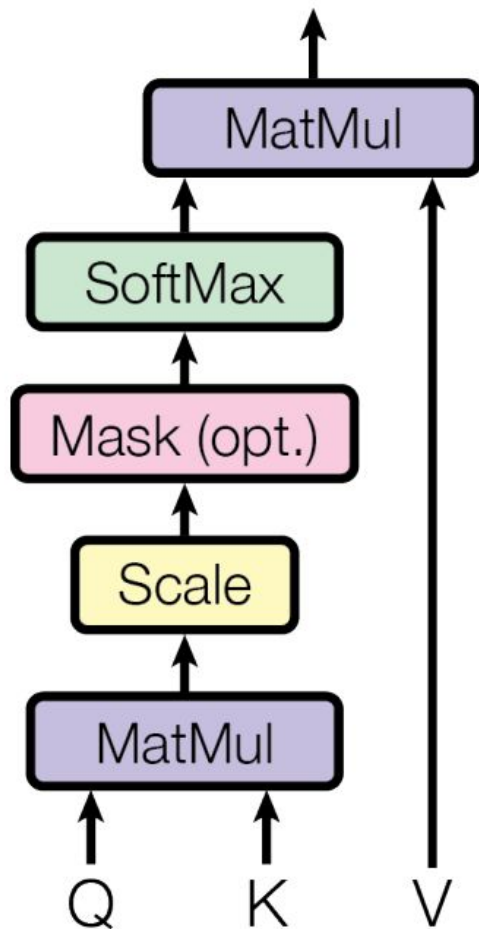


The diagram illustrates the Scaled Dot-Product Attention formula. It features a purple 2x3 matrix labeled Q , an orange 3x2 matrix labeled K^T , and a blue 2x3 matrix labeled V . The Q and K^T matrices are multiplied together, and the result is divided by $\sqrt{d_k}$. This entire operation is enclosed in a large parentheses, with the word "softmax" written to its left. To the right of the parentheses is the blue V matrix, followed by an equals sign and a pink 2x3 matrix labeled Z .

$$\text{softmax} \left(\frac{Q \times K^T}{\sqrt{d_k}} \right) V = Z$$

Self-Attention

- Q-K 유사도 기반 스코어
- Attention Distribution
- Attention Vector

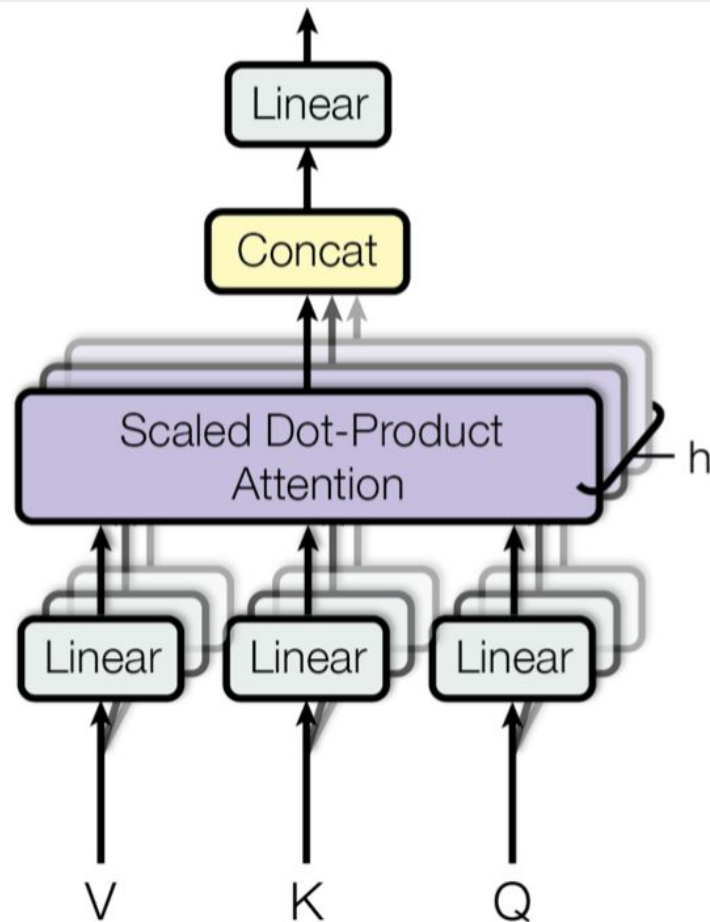


Multi-head Attention

단일 Attention으로는 다양한
문맥정보를 탐색할 수 없음

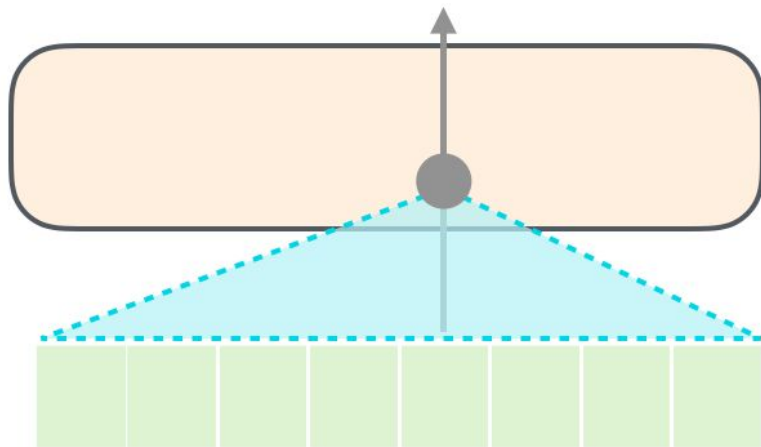
각 벡터를 N개로 분할해 각각
Attention 적용

이후 concat 후 linear를 통해 의미
통합

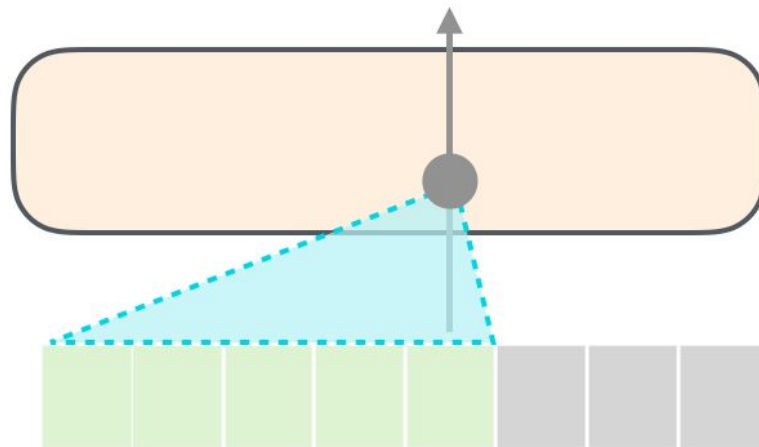


Self-Attention

Self-Attention

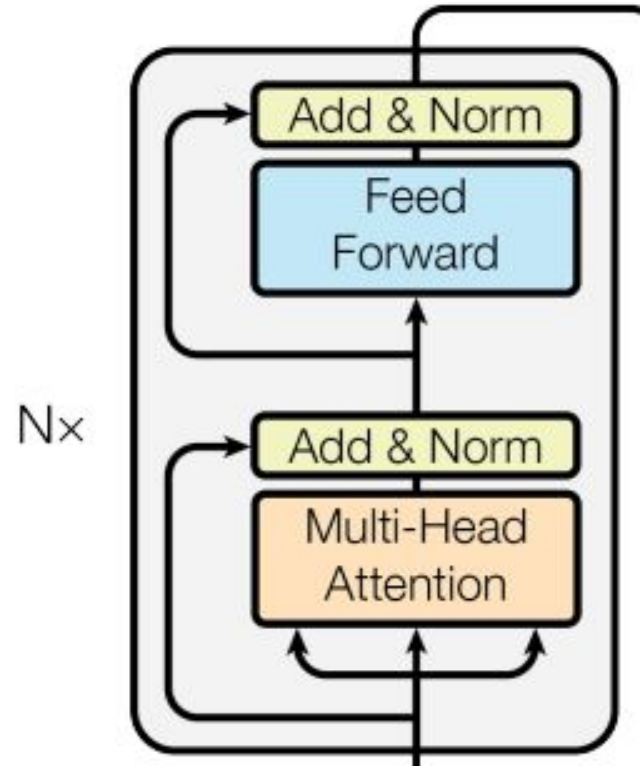


Masked Self-Attention



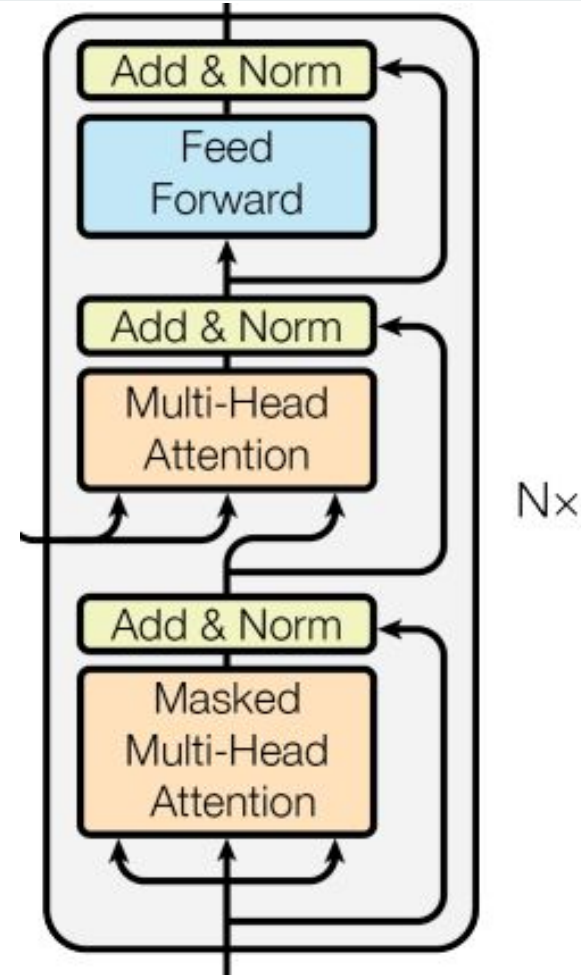
Encoder Block

- Multi-head Attention
- Residual connection
- Layer Normalization
- FFNN



Decoder Block

- Masked Multi-head Attention
- Residual connection
- Layer Normalization
- FFNN





Process

Encoder

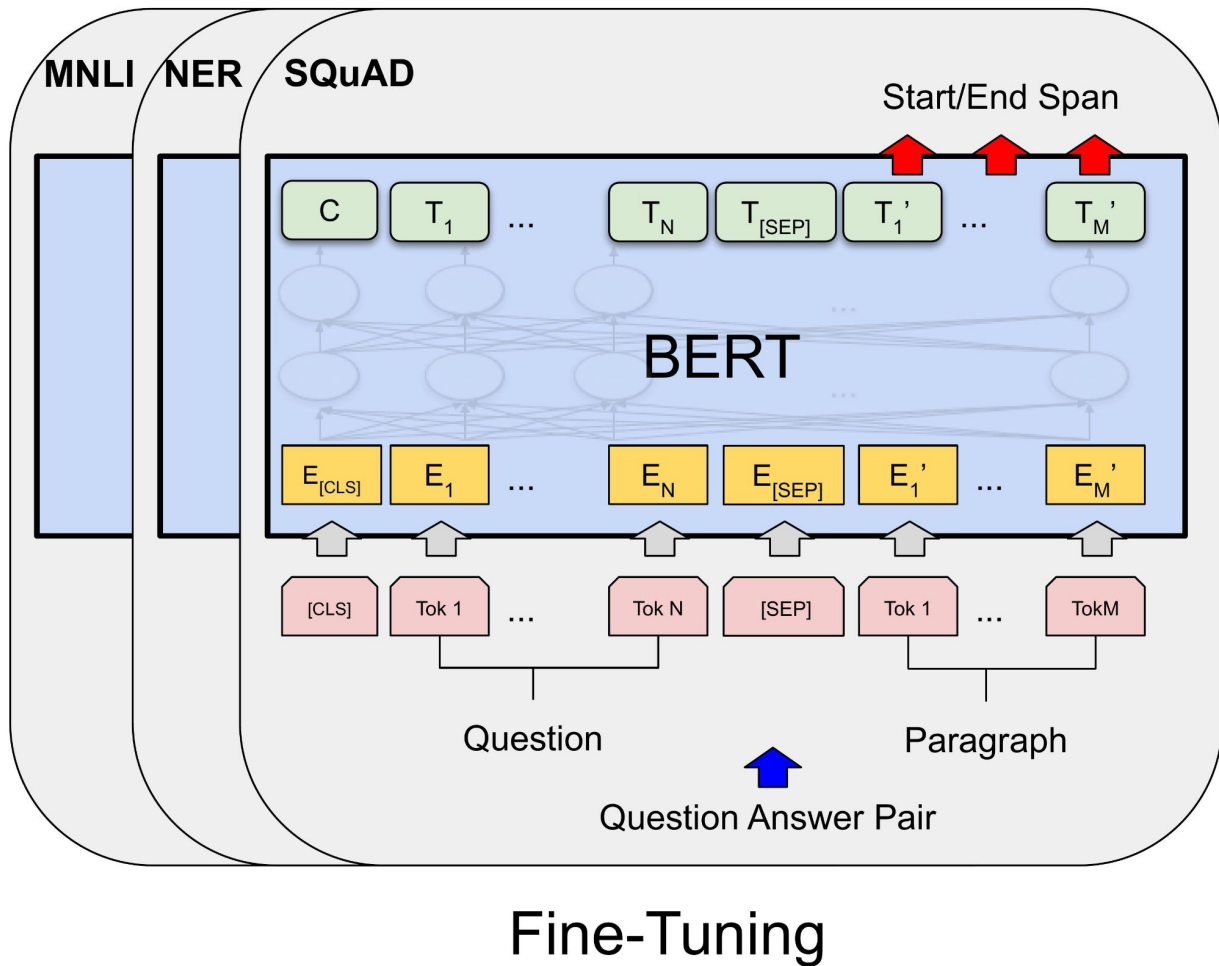
모든 정보 취합

Decoder

이전 스텝의 정보만
취합

BERT

Transformer의
Encoder 구조

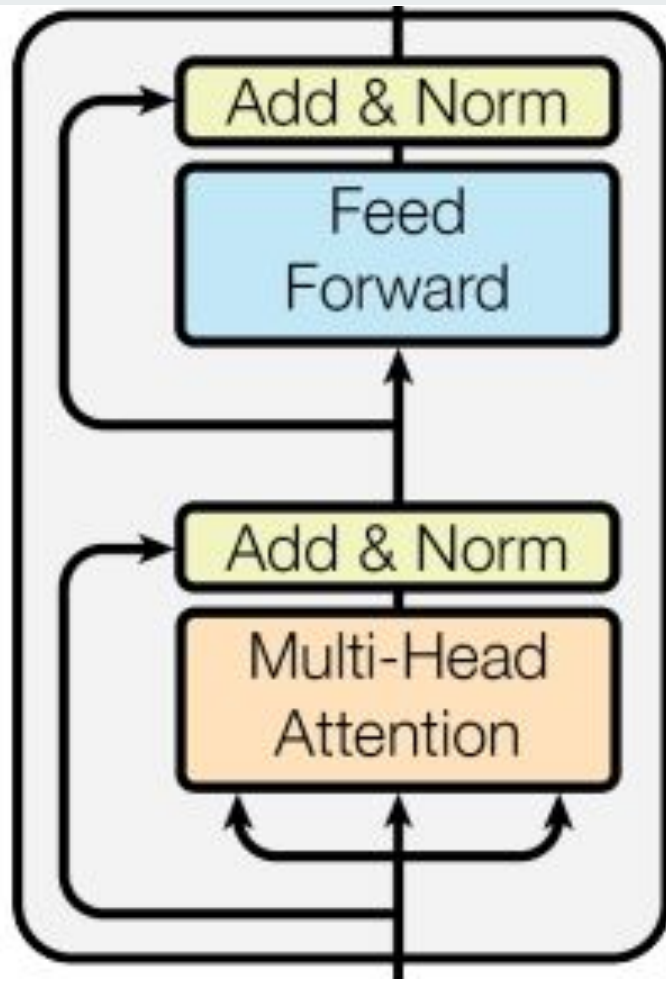


GPT

Masked Attention 사용

Transformer의 Decoder 구조 $N \times$

단, 인코더-디코더 어텐션 없음





GPT가 가져온 변화

- In-Context Learning
- LLM
- 생성형 AI



InstructGPT

InstructGPT란?

GPT-3 모델을 사용자의 지시(Instruction)에 더 잘 따르도록 Fine-tuning한 버전

Hugging Face에서 배포되는 모델은 -instruct, -chat, -rlhf와 같은 태그를 붙여 구분하기도 함



SFT(Supervised Fine-Tuning)

기존 모델은 “텍스트를 그럴듯하게 생성”하는 task

instruction-response 데이터로 “지시사항을 수행”하는 task를 학습



SFT의 한계

사람이 만든 예시로 데이터셋을 구성

- 데이터 수가 부족
- 모든 상황을 커버하지 못함
- 이 답이 좋은지 나쁜지 평가하는 기준이 없음



PPO(Proximal Policy Optimization)

응답에대한 사람의 선호를 대신 수행하는 Reward 모델을 이용해 더 높은 reward를 얻을 수 있도록 학습

- 더 자연스러운 문장 구조
- 더 정확한 정보
- 유해 정보 생성 억제

단, 모든 경우에 PPO가 필요한 것은 아님



PPO 학습 순서

1. 입력 x 에 대해 응답 후보 y 를 여러개 샘플링
2. 각 x, y 쌍에 대해 reward 모델로 reward 계산
3. KL penalty로 원래 policy와 너무 크게 차이나지 않도록 reward 값을 제한(Clipping)
4. reward가 높았던 출력의 확률을 높이고 reward가 낮은 출력의 확률을 낮추는 방향으로 업데이트